33. TLS Certificates and Trust Stores (ops view)

33. TLS Certificates and Trust Stores (ops view)

Phase: Phase 9 — Security and SSH
Estimated time: 1–2 days
Slug: `33-tls-certs`

Learning objectives

- Explain cert, key, chain, CA
- Inspect certs with openssl
- Know where system trust stores live
- Plan renewals before expiry

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Almost every service you run speaks TLS: HTTPS sites, APIs, databases, mail, service meshes. The most common TLS outage is boring: **a certificate expired** and nobody noticed. The second most common is a wrong hostname or a missing intermediate certificate. With `openssl` you can inspect any certificate or live server in seconds, and you can create test certificates for labs.

Core concepts

Term	Meaning
private key	Secret; never leaves the server; file mode 600
CSR	Certificate signing request: public key plus identity, sent to a CA
certificate	Signed statement binding a public key to names (SAN)
CA	Certificate authority that signs; clients trust a set of root CAs
chain	Leaf cert, intermediate(s), root; server must send leaf + intermediates
SAN	Subject Alternative Name list; hostnames the cert is valid for
notBefore/notAfter	Validity window; check with `-dates`
PEM / DER	Text (base64) vs binary encodings

```
client                                  server
  | ---- ClientHello (SNI: example.com) -->|
  | <--- ServerHello + cert chain ---------|
  |  verify: signature chain to trusted root
  |          hostname matches SAN
  |          now between notBefore..notAfter
  | ---- key exchange, encrypted data ---->|
```

The flag you will use daily is `-dates`, which prints `notBefore` and `notAfter` for a certificate.

Worked examples

Create a throwaway self-signed certificate for lab use:

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 \
  -keyout lab.key -out lab.crt \
  -subj "/CN=lab.local" -addext "subjectAltName=DNS:lab.local"
chmod 600 lab.key
```

Show validity dates, subject, issuer, and SAN:

```
openssl x509 -in lab.crt -noout -dates
openssl x509 -in lab.crt -noout -subject -issuer
openssl x509 -in lab.crt -noout -ext subjectAltName
```

Sample `-dates` output:

```
notBefore=Oct 21 08:00:00 2026 GMT
notAfter=Nov 20 08:00:00 2026 GMT
```

Look at a live server's certificate:

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/
null \
  | openssl x509 -noout -dates -issuer -subject
```

See the whole chain and verification result:

```
openssl s_client -connect example.com:443 -servername example.com -showcerts </
dev/null
# look for: Verify return code: 0 (ok)
```

Will it expire within 14 days? (exit code 0 means still valid after N seconds)

```
openssl x509 -in lab.crt -noout -checkend $((14*24*3600)) \
  && echo "ok for 14 days" || echo "expires soon"
```

Check that a key matches a certificate (hashes must be equal):

```
openssl x509 -in lab.crt -noout -pubkey | openssl sha256
openssl pkey -in lab.key -pubout | openssl sha256
```

Create a CSR for a real CA:

```
openssl req -new -newkey rsa:2048 -nodes -keyout site.key -out site.csr \
  -subj "/CN=www.example.org"
openssl req -in site.csr -noout -text | head -n 20
```

Test with curl, including a custom CA:

```
curl -v https://example.com 2>&1 | grep -E 'subject|expire|issuer'
curl --cacert lab.crt --resolve lab.local:8443:127.0.0.1 https://lab.local:8443/
```

Quick local TLS server for tests:

```
openssl s_server -accept 8443 -cert lab.crt -key lab.key -www
```

Mini-lab

1. Generate the lab certificate above with a 2-day lifetime.
2. Print dates with `-dates` and convert `notAfter` to epoch with `date -d`.
3. Run `-checkend 86400` and `-checkend 259200`; explain both exit codes.
4. Start `s_server` and connect with `s_client`. Find `Verify return code` and explain why it is not 0 for a self-signed cert.
5. Check the real `notAfter` of two public sites and write a small loop that prints `host days_left` for each.

```
for h in example.com openai.com; do
  end=$(echo | openssl s_client -connect $h:443 -servername $h 2>/dev/null \
        | openssl x509 -noout -enddate | cut -d= -f2)
  echo "$h $(( ( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ))"
done
```

Decision guide / failure modes

Symptom	Likely cause	Check
Browser: expired	notAfter passed	`-dates`
Hostname mismatch	name not in SAN	`-ext subjectAltName`
Works in browser, fails in curl/Java	missing intermediate	`s_client -showcerts`
Unknown CA	private CA not trusted	add to trust store or `--cacert`
Server will not start	key does not match cert	compare pubkey hashes
Random clients fail	wrong clock	`timedatectl`, chrony

Failure modes:

- Renewing the file but not reloading the service; the old cert stays in memory.
- Putting the private key in git or mode 644.
- Forgetting `-servername`, so a multi-site server returns the default cert.
- Automating issuance (ACME/certbot) without monitoring; always alert at 14 and 7 days before expiry.
- Using `-k` or `verify=False` “temporarily” and shipping it.

Common mistakes

- Copying private keys into git
- Disabling TLS verification permanently
- Forgetting intermediate chain

Exercises

Work in `exercises/33-tls-certs/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/33-tls-certs
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `openssl_show_dates_flag` in `exercise.sh` so it prints the option for `openssl x509` that outputs the `notBefore` and `notAfter` validity dates of a certificate. Print only the option with its leading dash. After the test passes, generate a short-lived self-signed certificate as shown in the lesson, run the option against it, and note both dates plus how many days remain until `notAfter`. Keep the function body to one line; the goal is recall of the lesson

vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/33-tls-certs/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is a CA?
2. How inspect expiry?
3. Why automate renewal?

Further reading

- https://www.openssl.org/docs/